Playwright Automation — 30-Day Complete Study Guide

Version: 1.0 · **Stack:** Playwright Test (JavaScript/TypeScript) · **Audience:** Beginners to intermediate QA / SDET

How to use this guide

1. Create a single lab repository (`pw-30-day-lab`) and commit after each day.
2. Run tests in UI mode once per day: `npx playwright test --ui` .
3. Keep a **flake log**: every flaky test → root cause → fix (usually wrong locator or missing wait).
4. Answers are included for **all** important questions and **all** mini-projects; still type code yourself.

Prerequisites

- Node.js 18+ and npm
- Basic JavaScript (async/await, imports)
- Optional: TypeScript for Days 7+

Quick reference

Command	Purpose
`npx playwright test`	Run all tests
`npx playwright test file.spec.ts`	Run one file
`npx playwright test --grep @smoke`	Filter by tag
`npx playwright test --headed`	See browser
`npx playwright test --debug`	Inspector
`npx playwright show-report`	HTML report

30-Day Study Schedule (Realistic)

Week	Focus	Hours/week
1	Setup, locators, assertions, forms	~14h
2	Waits, network, frames, files, API	~14h
3	Auth, fixtures, POM, data-driven	~14h
4	Visual, mobile, CI, capstone	~16h

Daily rhythm (≈2–2.5h weekdays, 3–4h weekend):

1. **30 min** — Read concepts + revision cards
2. **45 min** — Type examples in your lab repo (no copy-paste only)
3. **30 min** — Answer important questions aloud or in notes

4. **45 min** — Mini-project / extend capstone

Weekly checkpoints:

- **End Week 1:** 10+ tests on public demo sites; zero `waitForTimeout` in new tests
- **End Week 2:** 1 suite with `route` mock + download assertion
- **End Week 3:** POM for 2 pages + `storageState` auth
- **End Week 4:** CI workflow green + final LMS capstone

Suggested calendar

Day	Topic	Lab
1	Why Playwright & Environment Setup	2.0h
2	Test Anatomy, Fixtures & Basic Navigation	2.0h
3	Locators: getByRole & Accessible Queries	2.5h
4	CSS, XPath, test ids & Custom Locators	2.0h
5	Assertions & expect Polling	2.5h
6	Hooks, Tags & Test Organization	2.0h
7	playwright.config.ts Deep Dive	2.5h
8	Debugging: UI Mode, Trace Viewer, Inspector	2.0h
9	Filling Forms: text, password, textarea	2.0h
10	Checkboxes, Radios, Selects & Custom dropdowns	2.5h
11	Waiting Strategies	2.5h
12	Network: intercept, mock, HAR	2.5h
13	Frames & iFrames	2.5h
14	Multiple tabs & popups	2.5h
15	Dialogs: alert, confirm, prompt	2.5h
16	Downloads & uploads	2.5h
17	Screenshots & videos	2.5h
18	API testing with request fixture	2.5h
19	Authentication patterns	2.5h
20	Fixtures: extend test	2.5h
21	Page Object Model (POM)	2.5h
22	Data-driven tests	2.5h
23	Parallelism & sharding	2.5h

24	Visual regression	2.5h
25	Mobile & device emulation	2.5h
26	Component testing (intro)	2.5h
27	CI: GitHub Actions	3.0h
28	Reporting & Allure	2.5h
29	Performance & stability	2.5h
30	Capstone: Full E2E suite	4.0h

Day 1: Why Playwright & Environment Setup

Estimated time: 2.0 hours

Learning objectives

- Understand what Playwright is and when to use it
- Install Node.js, VS Code, and Playwright Test
- Run your first passing test

Key concepts (simple explanations)

End-to-end (E2E) testing — Automating a real browser to click, type, and verify UI behavior like a user would.

Playwright Test runner — Built-in test framework with fixtures, parallel runs, retries, traces, and HTML reports.

Cross-browser — Chromium, Firefox, and WebKit from one API; projects in config map to browsers.

Auto-waiting — Playwright waits for elements to be actionable before acting, reducing flaky sleeps.

Hands-on example

```
# New project
mkdir pw-study && cd pw-study
npm init -y
npm init playwright@latest
# Choose: TypeScript or JavaScript, tests folder, GitHub Actions optional

npx playwright test
npx playwright test --ui
```

```
// tests/example.spec.js
import { test, expect } from '@playwright/test';

test('homepage has title', async ({ page }) => {
  await page.goto('https://playwright.dev');
```

```
  await expect(page).toHaveTitle(/Playwright/);
});
```

Important questions (with answers)

Q: What problem does Playwright solve compared to manual QA?

A: It repeats consistent user flows quickly, catches regressions early, and runs in CI on every commit.

Q: Name three browsers Playwright supports out of the box.

A: Chromium, Firefox, and WebKit.

Q: What is the difference between `page.click` and a raw DOM click in brittle scripts?

A: Playwright auto-waits for visibility, stability, and enabled state; raw clicks often race the UI.

Revision notes

- Install: `npm init playwright@latest`
- Run: `npx playwright test`, debug: `--ui`, `--debug`
- Test file pattern: `*.spec.ts` / `*.spec.js`
- Fixtures: `page`, `context`, `browser` injected by test runner

Real-time mini-project: Mini-Project 1: Smoke test your favorite public site

Write one test that opens a site, checks the title, and asserts one visible heading.

Answer / solution:

```
import { test, expect } from '@playwright/test';

test('site smoke', async ({ page }) => {
  await page.goto('https://example.com');
  await expect(page).toHaveTitle(/Example Domain/);
  await expect(page.getByRole('heading', { level: 1 })).toBeVisible();
});
```

Day 2: Test Anatomy, Fixtures & Basic Navigation

Estimated time: 2.0 hours

Learning objectives

- Read Playwright test structure: describe, test, hooks
- Use `page.goto`, `page.url`, navigation options
- Understand test isolation (fresh context per test)

Key concepts (simple explanations)

test vs test.describe — `test` is one case; `describe` groups related cases and shared hooks.

async/await — Every browser action returns a Promise; always await Playwright calls.

baseURL — Set in config so `goto('/login')` resolves to your app root.

test.info() — Metadata like title, project, retry index for logging and attachments.

Hands-on example

```
import { test, expect } from '@playwright/test';

test.describe('Navigation', () => {
  test.beforeEach(async ({ page }) => {
    await page.goto('/');
  });

  test('lands on home', async ({ page }) => {
    await expect(page).toHaveURL(/\/$/);
  });
});
```

Important questions (with answers)

Q: Why is each test isolated?

A: Fresh browser context prevents cookie/storage leakage between tests.

Q: What does `waitUntil: 'networkidle'` do?

A: Waits until no network connections for ~500ms (use sparingly).

Q: When use `page.goto` vs `page.reload`?

A: `goto` for new URL; `reload` to refresh current page.

Revision notes

- Default timeout: action 0 uses expect timeout (often 5s)
- Prefer `toHaveURL` over manual string compares
- Use `test.describe` + `beforeEach` for shared setup

Real-time mini-project: Mini-Project 2: Multi-page navigation

Test that clicking 'More information' on example.com opens the IANA page.

Answer / solution:

```
import { test, expect } from '@playwright/test';

test('follow link', async ({ page }) => {
  await page.goto('https://example.com');
  await page.getByRole('link', { name: 'More information...' }).click();
  await expect(page).toHaveURL(/iana\.org/);
});
```

Day 3: Locators: getByRole & Accessible Queries

Estimated time: 2.5 hours

Learning objectives

- Prefer user-facing locators (role, label, text)
- Chain and filter locators
- Understand strict mode (single match)

Key concepts (simple explanations)

getByRole — Maps to ARIA roles: button, link, textbox, heading — mirrors how assistive tech sees the page.

getByLabel — Finds inputs associated with a or aria-label.

getByPlaceholder / getByText — Fallbacks when roles are weak; text should be stable.

locator.first() / nth() — Escape hatch when multiple matches; fix UI duplication when possible.

Hands-on example

```
await page.getByRole('button', { name: 'Sign in' }).click();
await page.getByLabel('Email').fill('user@example.com');
await page.getByRole('heading', { name: 'Dashboard' }).waitFor();
```

Important questions (with answers)

Q: Why prefer getByRole over CSS #id?

A: IDs change; roles reflect user-meaningful elements and survive refactors better.

Q: What error does strict mode cause?

A: Playwright throws if a locator resolves to more than one element.

Q: How to click the second 'Edit' button?

A: Use `.nth(1)` or filter by parent row: `row.getByRole('button', { name: 'Edit' })`.

Revision notes

- Order: Role → Label → Placeholder → Text → Test id
- Use `{ name: '...' }` exact or regex
- Debug: `await page.getByRole('button').all()` in UI mode

Real-time mini-project: Mini-Project 3: Login form locators (practice site)

Automate login on https://the-internet.herokuapp.com/login (student / SuperSecretPassword!).

Answer / solution:

```
import { test, expect } from '@playwright/test';

test('heroku login', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com/login');
  await page.getByLabel('Username').fill('student');
  await page.getByLabel('Password').fill('SuperSecretPassword!');
  await page.getByRole('button', { name: 'Login' }).click();
```

```
  await expect(page.getByText('You logged into a secure area!')).toBeVisible();
});
```

Day 4: CSS, XPath, test ids & Custom Locators

Estimated time: 2.0 hours

Learning objectives

- Use data-testid responsibly
- Combine locators with and/or
- Build reusable locator helpers

Key concepts (simple explanations)

page.locator('css') — Powerful but brittle; use for shadow DOM or legacy apps.

getByTestId — Reads `data-testid` ; contract between dev and QA.

filter() — Narrows locators: `page.getByRole('listitem').filter({ hasText: 'Due' })` .

locator.or() — Try alternative locators when UI varies.

Hands-on example

```
const row = page.getByRole('row').filter({ hasText: 'Invoice #42' });
await row.getByRole('button', { name: 'Pay' }).click();
```

Important questions (with answers)

Q: When is CSS acceptable?

A: Shadow roots, canvas, or no semantic HTML — document why.

Q: What is `data-testid` convention?

A: Stable hook not tied to styling; e.g. `data-testid=checkout-submit` .

Q: Difference filter vs has?

A: `filter({ has: locator })` requires child locator inside.

Revision notes

- testid config: `testIdAttribute: 'data-test-id'`
- Avoid XPath unless unavoidable

Real-time mini-project: Mini-Project 4: Table row action

On a demo table page, click action in a specific row (use filter/hasText).

Answer / solution:

```
import { test, expect } from '@playwright/test';
```

```
test('sortable table row action', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com/tables');
  const table = page.getByRole('table');
  const target = table.getByRole('row').filter({ hasText: 'Action' }).first();
  await expect(target).toBeVisible();
  await target.getByRole('link', { name: 'edit' }).click();
  await expect(page).toHaveURL(/tables/);
});
```

Day 5: Assertions & expect Polling

Estimated time: 2.5 hours

Learning objectives

- Use web-first assertions (auto-retry)
- Soft assertions when collecting multiple checks
- Negation and custom messages

Key concepts (simple explanations)

expect(locator).toBeVisible() — Retries until timeout; preferred over manual wait + assert.

toHaveText / toContainText — String or regex; whitespace can matter — use regex for flexibility.

expect.poll() — Custom async polling for non-locator state.

expect.soft() — Continues test after failure; useful for audit-style checks.

Hands-on example

```
await expect(page.getByRole('alert')).toHaveText('Saved');
await expect(page.getByTestId('score')).toHaveText(/\d+/);
await expect.poll(async () => {
  const r = await page.request.get('/api/status');
  return r.ok();
}).toBeTruthy();
```

Important questions (with answers)

Q: Why not use `expect(await el.isVisible()).toBe(true)` ?

A: No retry — race conditions cause flakes.

Q: Default expect timeout?

A: 5 seconds (configurable via `expect: { timeout }`).

Q: When use soft assertions?

A: Bulk UI validation where you want all failures in one run.

Revision notes

- Matchers: visible, enabled, checked, value, attribute, count

- Use `not` for negative cases

Real-time mini-project: Mini-Project 5: Assert dashboard widgets

After login, assert 3 widgets visible with expected labels.

Answer / solution:

```
import { test, expect } from '@playwright/test';

test('dashboard widgets after login', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com/login');
  await page.getByLabel('Username').fill('student');
  await page.getByLabel('Password').fill('SuperSecretPassword!');
  await page.getByRole('button', { name: 'Login' }).click();
  await expect(page.getByText('You logged into a secure area!')).toBeVisible();
  await expect.soft(page.getByRole('heading', { level: 2 })).toHaveText('Secure
Area');
  await expect.soft(page.getByRole('link', { name: 'Logout' })).toBeVisible();
});
```

Day 6: Hooks, Tags & Test Organization

Estimated time: 2.0 hours

Learning objectives

- Use beforeEach / afterEach for shared setup and teardown
- Apply test.describe for logical grouping
- Run subsets with tags and --grep
- Skip or mark broken tests with test.skip and test.fixme

Key concepts (simple explanations)

beforeEach — Runs before every test in the describe block — ideal for navigation to a starting page.

afterEach — Cleanup: delete test data, reset mocks; runs even if test fails.

test.describe.configure — Set mode: parallel vs serial when tests share state.

Annotations @smoke — Tags in title or test.info().annotations; filter in CI for fast smoke runs.

Hands-on example

```
import { test, expect } from '@playwright/test';

test.describe('Internet app', () => {
  test.beforeEach(async ({ page }) => {
    await page.goto('https://the-internet.herokuapp.com');
  });

  test('home @smoke', async ({ page }) => {
    await expect(page.getByRole('heading', { level: 1 })).toHaveText('Welcome');
```

```
  });

  test.fixme('broken feature', async () => {
    // skipped until bug-123 fixed
  });
});
// Run: npx playwright test --grep @smoke
```

Important questions (with answers)

Q: When should tests run serially?

A: When they mutate shared backend state or depend on order (prefer isolated data instead).

Q: Difference between test.skip and test.fixme?

A: `skip` is intentional; `fixme` signals known product/test bug to fix later.

Q: Can hooks be async?

A: Yes — always await async setup (DB seed, login).

Q: How to run only one file?

A: `npx playwright test tests/login.spec.ts`

Q: Why group with describe?

A: Shared hooks, readable reports, and scoped configuration.

Revision notes

- Structure: `describe` → `beforeEach` → `test`
- Tag smoke tests for PR checks
- Avoid serial unless necessary
- Name tests: behavior + expected outcome

Real-time mini-project: Organize Herokuapp smoke suite

Create a describe block with 3 tagged tests: headings, links, and footer.

Answer / solution:

```
import { test, expect } from '@playwright/test';

test.describe('Heroku smoke @smoke', () => {
  test.beforeEach(async ({ page }) => {
    await page.goto('https://the-internet.herokuapp.com');
  });

  test('title @smoke', async ({ page }) => {
    await expect(page).toHaveTitle(/The Internet/);
  });

  test('examples link @smoke', async ({ page }) => {
    await expect(page.getByRole('link', { name: 'Form Authentication'
```

```
})).toBeVisible();
  });

  test('powered by @smoke', async ({ page }) => {
    await expect(page.getByText('Powered by Elemental Selenium')).toBeVisible();
  });
});
```

Day 7: playwright.config.ts Deep Dive

Estimated time: 2.5 hours

Learning objectives

- Configure baseURL, timeouts, and reporters
- Define browser projects (chromium, firefox, webkit)
- Set retries and trace for CI
- Understand `use` vs project-level overrides

Key concepts (simple explanations)

defineConfig — Central place for testDir, workers, globalSetup, and expect defaults.

projects[] — Each project is a browser/device profile; same tests run per project.

use.baseURL — Enables `page.goto('/login')` without hardcoding host.

trace / screenshot / video — Artifacts on failure: `trace: 'on-first-retry'` balances speed and debuggability.

Hands-on example

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './tests',
  fullyParallel: true,
  forbidOnly: !!process.env.CI,
  retries: process.env.CI ? 2 : 0,
  reporter: [['html', { open: 'never' }], ['list']],
  use: {
    baseURL: 'http://localhost:3000',
    trace: 'on-first-retry',
    screenshot: 'only-on-failure',
  },
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'firefox', use: { ...devices['Desktop Firefox'] } },
  ],
});
```

Important questions (with answers)

Q: What does forbidOnly do in CI?

A: Fails build if `test.only` left in code — prevents accidental focused runs.

Q: How to run only chromium?

A: `npx playwright test --project=chromium`

Q: Where set global timeout?

A: `timeout: 60000` at config root (per test).

Q: ENV in config?

A: Use `process.env.BASE_URL` for staging vs prod.

Q: workers locally vs CI?

A: Local: CPU cores; CI: often 2–4 to avoid OOM.

Revision notes

- Never commit secrets in config — use env vars
- baseURL + relative goto = portable tests
- HTML report: `npx playwright show-report`
- Match retries to pipeline stability goals

Real-time mini-project: Config lab: multi-project

Add a `smoke` project that only runs tests matching @smoke (use grep in npm script).

Answer / solution:

```
// package.json scripts
{
  "scripts": {
    "test": "playwright test",
    "test:smoke": "playwright test --grep @smoke --project=chromium"
  }
}
```

Day 8: Debugging: UI Mode, Trace Viewer, Inspector

Estimated time: 2.0 hours

Learning objectives

- Debug failing tests with --debug and Inspector
- Use UI Mode to pick locators and watch steps
- Read traces: actions, network, console, snapshots

Key concepts (simple explanations)

UI Mode — Interactive runner: time travel, watch locators, edit and re-run.

Inspector — Step-through with highlight; opens with PWDEBUG=1 or --debug.

Trace Viewer — Post-mortem timeline; open zip from report or `show-trace`.

page.pause() — Temporary breakpoint in code during local debug only.

Hands-on example

```
npx playwright test tests/login.spec.ts --ui
npx playwright test --debug
npx playwright show-trace test-results/.../trace.zip
```

Important questions (with answers)

Q: First step when a test fails in CI?

A: Download trace from artifact → find failing action → fix locator/wait.

Q: What is actionability?

A: Element visible, stable, enabled, not obscured — shown in trace.

Q: How to log locator count?

A: `console.log(await page.getByRole('button').count())` in headed mode.

Q: Danger of page.pause()?

A: Never merge to main — blocks CI.

Q: UI Mode vs debug?

A: UI Mode for exploration; debug for stepping one test.

Revision notes

- Re-run single test: `-g "test name"`
- Trace on first retry catches flakes
- Use Pick Locator in UI mode
- Compare screenshot in trace to expected UI

Real-time mini-project: Debug a failing locator

Intentionally wrong locator, run --ui, fix using Pick Locator.

Answer / solution:

```
// Wrong on purpose — fix in UI mode
test('debug me', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com/login');
  await page.getByRole('button', { name: 'Log in' }).click(); // fix name to 'Login'
});
```

Day 9: Filling Forms: text, password, textarea

Estimated time: 2.0 hours

Learning objectives

- Use fill for inputs and textareas
- Tab order and keyboard navigation
- Clear and re-fill patterns

Key concepts (simple explanations)

fill() — Sets value directly — fastest and least flaky for standard inputs.

pressSequentially() — Character-by-character; use for inputs that validate per keystroke.

keyboard.press — Enter to submit, Tab to move focus, shortcuts with modifiers.

inputValue() — Read current value for assertions.

Hands-on example

```
await page.getByLabel('Username').fill('student');
await page.getByLabel('Password').fill('secret');
await page.keyboard.press('Enter');
await expect(page.getByLabel('Username')).toHaveValue('student');
```

Important questions (with answers)

Q: fill vs type (deprecated)?

A: Prefer fill; type was removed in favor of pressSequentially.

Q: How to append text?

A: read value then fill combined, or click end + pressSequentially.

Q: Contenteditable?

A: Use locator.fill if supported or keyboard typing.

Q: Password fields?

A: Same as text; never log values in CI output.

Q: Disabled field?

A: Action fails — assert toBeDisabled first.

Revision notes

- getByLabel for form fields
- Submit with button click or Enter
- Assert with toHaveValue
- Avoid focusing via brittle CSS

Real-time mini-project: Registration form

Fill all fields on https://demoqa.com/automation-practice-form (sample subset).

Answer / solution:

```
import { test, expect } from '@playwright/test';

test('practice form partial', async ({ page }) => {
  await page.goto('https://demoqa.com/automation-practice-form');
  await page.getByPlaceholder('First Name').fill('Ada');
  await page.getByPlaceholder('Last Name').fill('Lovelace');
  await page.getByPlaceholder('name@example.com').fill('ada@example.com');
  await expect(page.getByPlaceholder('First Name')).toHaveValue('Ada');
});
```

Day 10: Checkboxes, Radios, Selects & Custom dropdowns

Estimated time: 2.5 hours

Learning objectives

- check/uncheck and expect checked state
- selectOption on native selects
- Click combobox + option for custom UI

Key concepts (simple explanations)

check() / uncheck() — For checkbox inputs; asserts state with toBeChecked.

selectOption — By value, label, or index on